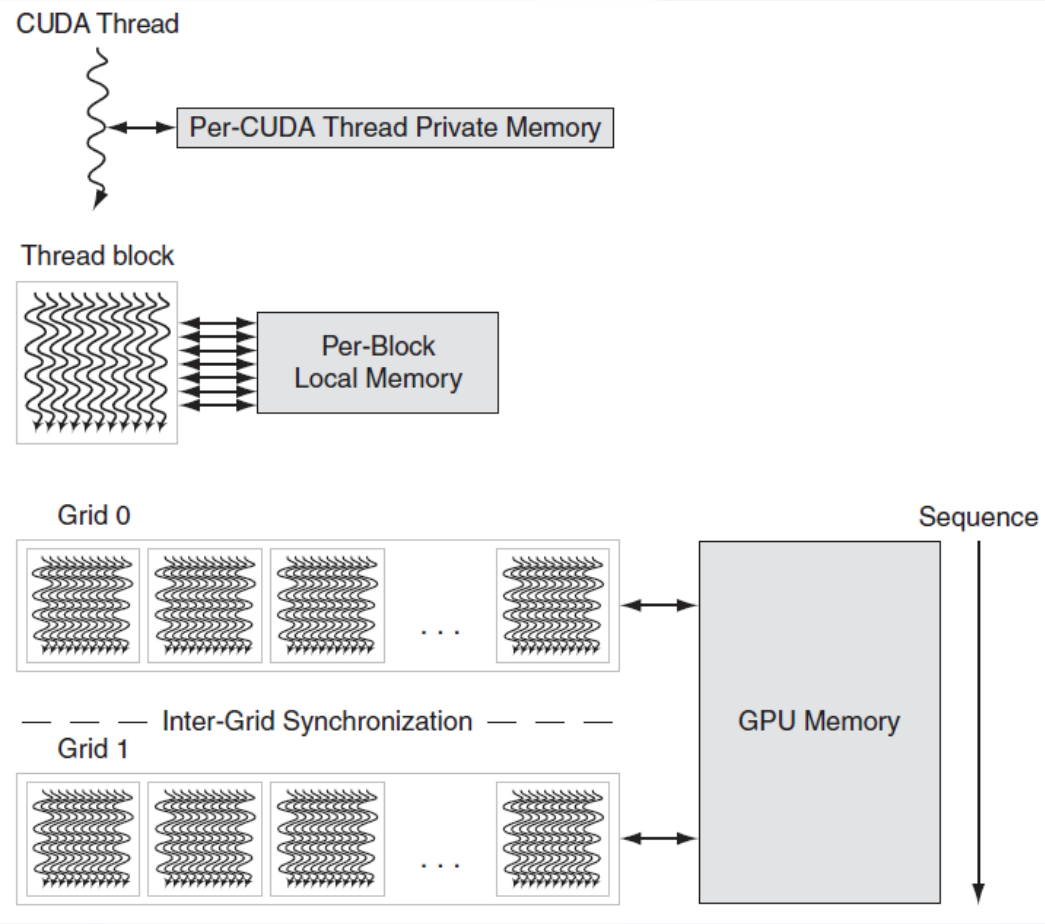
Memory Model

- Related to thread model



描述	CUDA名称	OpenCL名称
所有线程（或所work-items）均可访问	global memory	global memory
只读存储器	constant memory	constant memory
线程块（或work-group）内部线程访问	shared memory	local memory
单个线程（或work-item）可以访问	local memory	private memory

Memory Model

- TLP is decided by
  - # threads
  - # thread blocks
  - Size of register file
  - Size of shared memory

Example:

GPU	Volta GV100
Compute Capability	7.0
Threads / Warp	32
Max Warps / SM	64
Max Threads / SM	2048
Max Thread Blocks / SM	32
Max 32-bit Registers / SM	65536
Max Registers / Block	65536
Max Registers / Thread	255[1]
Max Thread Block Size	1024
FP32 Cores / SM	64
Ratio of SM Registers to FP32 Cores	1024
Shared Memory Size / SM	Configurable up to 96 KB

V100, 256KB RF, 96KB shared memory, 2048 threads per SM

256KB RF / 2048 threads = 32 regs per thread
More regs needed, then less threads

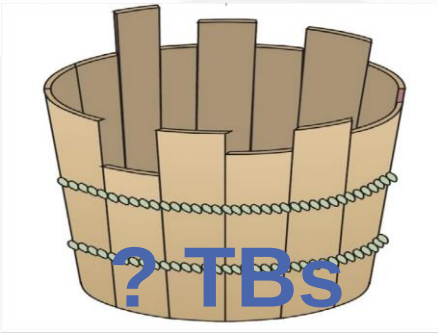
96KB shm / 32 blocks = 3KB per block
More shm needed, then less blocks

One kernel with 64 regs/thread and 8KB shm, 128 threads/TB

256KB RF / 64 reg/thread = 1024 threads, i.e. 8 TBs
96KB shm / 8KB shm = 12 TBs

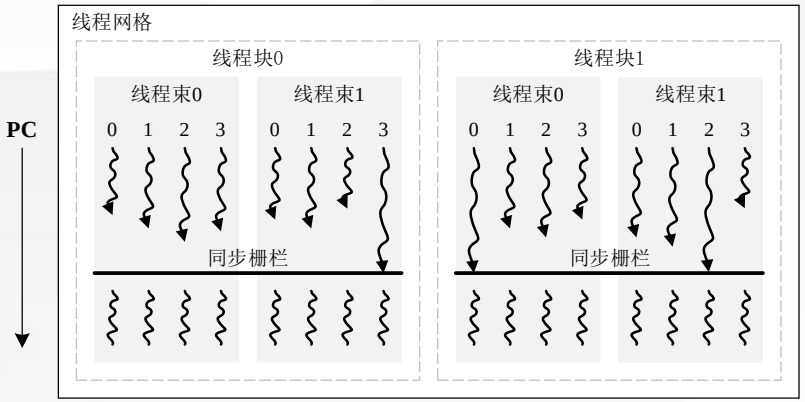
reg限制: 1024/128=8

shared memory 限制: 12



Synchronization

- Sync between threads in a block scope



__syncthreads()及其对应的bar指令原理

```
1   __global__ void  block_mul(float* d_A, float* d_B, float* d_C)
2   {
3       __shared__ float Mds[BLOCK_SIZE][BLOCK_SIZE];
4       __shared__ float Nds[BLOCK_SIZE][BLOCK_SIZE];
5       int row = threadIdx.x + blockIdx.x * blockDim.x;
6       int col = threadIdx.y + blockIdx.y * blockDim.y;
7       int tx = threadIdx.x; int ty = threadIdx.y;
8       float P = 0;
9       for(int i = 0; i < N/BLOCK_SIZE; i++)
10      {
11          Mds[tx][ty] = d_A[row * N + ty + i * BLOCK_SIZE];
12          Nds[tx][ty] = d_B[col + (tx + i * BLOCK_SIZE) * K];
13          __syncthreads();
14          for(int j = 0; j < BLOCK_SIZE; j++)
15          {
16              P += Mds[tx][j] * Nds[j][ty];
17              __syncthreads();
18          }
19       }
20       d_C[row * K + col] = P;
21  }
```

- Other scopes
  - cudaDeviceSynchronize()
  - cudaStreamSynchronize()